

Section 2: EEP1 Interrupt Mechanism

2026年3月14日 星期六 15:43

This section explores the additional interrupt hardware provided in EEP11 and how it can be used. To support interrupts, the EEP11 CPU (provided in the eep11 project) includes three additions relative to EEP1:

1. New hardware (shown in red in the control path, Figure 5) implementing an interrupt mechanism.

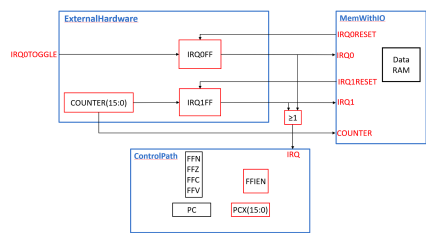
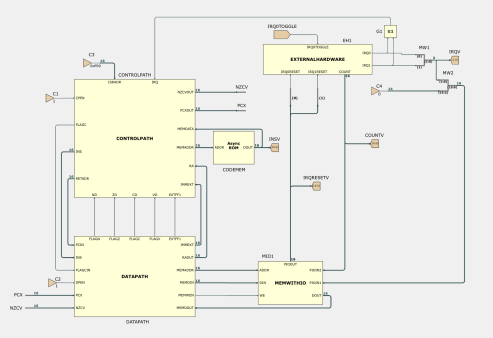


Figure 5: EEP11 Additional Hardware for Interrupts

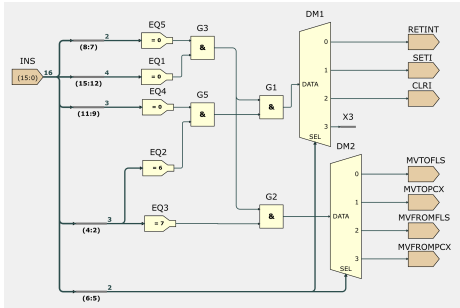


2. Additional instructions (Figure 7) allowing read/write access to the N,Z,C,V flags, interrupt-related registers, and return from an interrupt service routine.

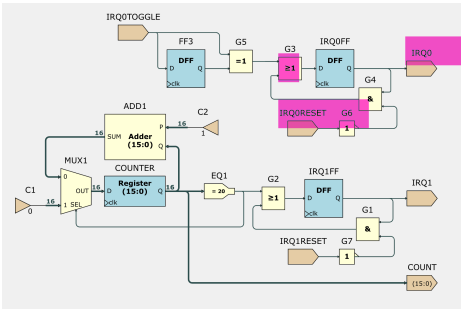
EEP1 "Extra MOV" Coding	INS (8)	N (4:2)	C (4:2)	A (11:9)	B (7:5)	INS Alternative Mnemonic	Operation	Notes
MOV R6, R6, R0	0	6	0	0	0	0x0018 RETINT	PC ← PCX; Flag := 1	Return from interrupt, enable interrupts
MOV R6, R6, R1	0	6	0	1	0	0x0038 SETI	Flag := 1	Allow (enable) interrupts
MOV R6, R6, R2	0	6	0	2	0	0x0058 CLRI	Flag := 0	Disable interrupts
MOV R7, R6, R0	0	7	a	0	0	MOV Flags, Ra	NZCV ← Ra(3:0)	Restore flags from register (no write to Ra)
MOV R7, R6, R1	0	7	a	1	0	MOV PCX, Ra	PCX ← Ra	Restore PCX from register (no write to Ra)
MOV R7, R6, R2	0	7	a	2	0	MOV Ra, Flags	Ra(3:0) ← NZCV	Save flags to register
MOV R7, R6, R3	0	7	a	3	0	MOV Ra, PCX	Ra ← PCX	Save PCX to register

Figure 7: EEP11 Instructions to support interrupts

So R6 & R7 can never be used in this case.



3. External hardware on sheet externalhardware. This provides two interrupt sources interfaced to EEP1 as shown in Figure 4:



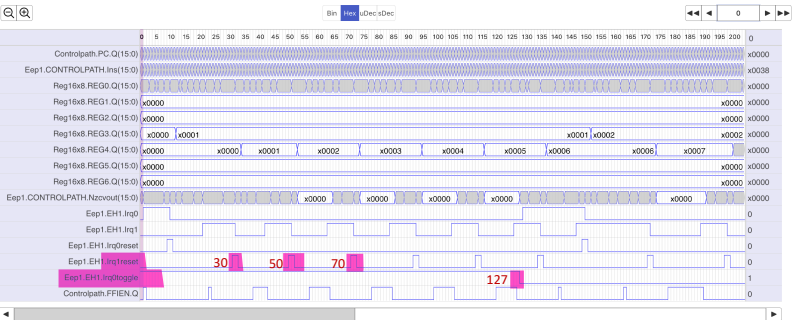
Address	Read 15:2	Read 1	Read 0	Write 15:2	Write 1	Write 0	Read Operation	Write Operation
0x7FFF	COUNT	1	0	n/a	Reset INT1	Reset INT0	Read INT1 counter value	1 → reset the corresponding interrupt request (IRQ1 or IRQ0)
0x7FFE	0	IRQ1	IRQ0				Read interrupt request status: 1 → set	n/a

Figure 4: EEP11 I/O Memory map

– Interrupt 0 (INT0): an interrupt request is generated whenever input IRQTOGGLE is toggled in the step simulator. The supplied eep1x design toggles IRQTOGGLE once every 128 cycles during simulation.

– Interrupt 1 (INT1): a periodic interrupt generated by a counter approximately every 20 clock cycles.

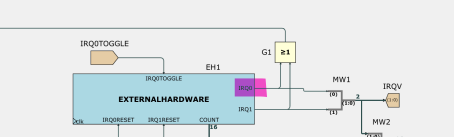
When an event occurs, IRQ0/1 is latched and remains high. If the software then issues a clear/reset signal, IRQ0/1RESET goes high, which clears the corresponding interrupt latch and drives IRQ0 or IRQ1 low again.



Which is true.

4.1 Using a polling loop to detect hardware status

This section examines the external interrupt-generating hardware. Interrupt 0 (INT0) originates from the externalhardware sheet. An interrupt request (IRQ0) is set whenever the IRQTOGGLE input changes state. INT0 can be cleared by EEP1 instructions that write to the Reset INT0 bit in memory-mapped I/O (Figure 4).





The program `iotest1.txt` uses the interrupt-status and reset bits from Figure 4 to detect INT0. The program repeatedly reads the IRQ0 status bit to determine whether the interrupt source is active. This technique is known as polling and contrasts with the interrupt-driven approach used in the next section.

```

iotest1.txt
LDR R0, [0xFFFF] // load IRQ status
AND R0, #1 // test IRQ0
JEQ -2 // loop if no IRQ0
MOV R0, #1
STR R0, [0xFFFF] // reset IRQ0
MOV R0, #0
STR R0, [0xFFFF] // stop reset
ADD R2, #1 // increment software counter
JMP -8

```

So according to the code, if $R0 = 1$, it will continue to loop.
If $R0 = 1$, we will reset it to be 0, and add 1 to R2. R2 helps to count total times of polling.

Task 3. Simulate the eep1 (not eep1x) sheet of lab3i with instruction memory loaded from `iotest1.ram` in the step simulator. Run a few cycles to observe the normal polling loop. Then manually toggle IRQ0TOGGLE and observe how this generates an IRQ0 request, which the program detects and clears, incrementing R2. The `iotest1.txt` program therefore counts edges on IRQ0TOGGLE using memory-mapped I/O and a software loop.

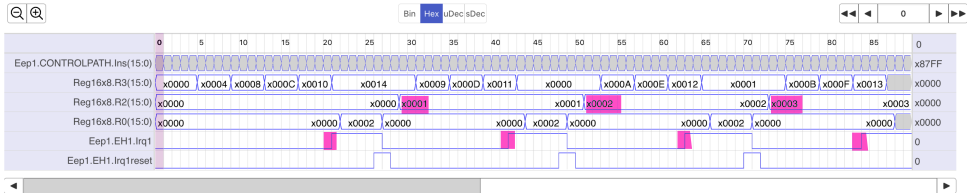
We can observe that when IRQ0TOGGLE doesn't change, eep1 keeps looping in 0, 1, 2 instructions, and R2 doesn't change at all.
As soon as we set IRQ0TOGGLE to 1, first eep1 will run to end of previous loop. After that, this time we jump out of the loop, and 8 cycles later R2 increments by 1.

Repeat the task using `iotest2.ram` in the waveform simulator. The program `iotest2.txt` detects interrupt source 1 (INT1), which repeatedly sets IRQ1 approximately every 20 clock cycles via a counter on the external-hardware sheet. View IRQ1 and IRQ1reset in addition to PC, INS, R0, and R2. The code is otherwise identical to `iotest1.txt`, so you should observe IRQ1 being detected and reset at this regular interval.

```

iotest2.txt
LDR R3, [0xFFFF] // load COUNT
LDR R0, [0xFFFF] // load IRQ status
AND R0, #1 // test IRQ1
JEQ -2
MOV R0, #1
STR R0, [0xFFFF] // reset IRQ1
MOV R0, #0
STR R0, [0xFFFF] // stop reset
ADD R2, #1 // increment software counter
JMP -8

```



We can observe that the logic is same as before. But now IRQ1 is set within the hardware to toggle every 20 cycles, so all of the changes happen at a period of 20.
When IRQ1 doesn't toggle, R3 increments by 4 during a loop; when IRQ1 toggles, R3 increments by 10.

4.2 Interrupt interface to hardware

The program `int-test1.txt` contains code starting at address 0 that enables interrupts (SETI) and then loops continuously incrementing R1. This loop is the base program. An Interrupt Service Routine (ISR), located at address 0xFF00, services the INT1 timer interrupt. Each time IRQ1 is asserted, EEP11 automatically jumps to the ISR at 0xFF00. The ISR resets the interrupt, increments R2, and then returns to the base program using RETINT, restoring the flags so that base program execution is unaffected. The interrupt vector address (0xFF00) is fixed in the EEP11 hardware.

```

int-test1.txt
// test interrupt 1
// base program
SETI // set PFIN_Q (enable interrupts)
LOOP ADD R0, #1 // increment R0 at 1/2 the rate of R0
JNE LOOP
// code from here goes into address 0xFF00
ISR
STR R0, [0xFF00] // save base program R0
MOV R0, #0 // save base program flags
STR R0, [0xFF00] // save base program R1
MOV R1, [0xFFFF] // Interrupt 1 reset high
STR R1, [0xFFFF] // Interrupt 1 reset low
STR R1, [0xFFFF] // Interrupt 1 reset low
// Payload for interrupt 1
// Increment R2 by 1
LDR R1, [0xFF00]
ADD R1, #1
STR R1, [0xFF00]
// end of payload
LDR R1, [0xFFFF] // restore base program R1
MOV R0, #0 // restore base program flags
LDR R0, [0xFF00] // restore base program R0
RETINT // return to base program and set PFIN

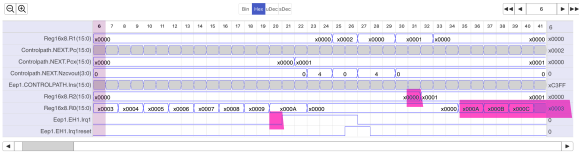
```

Task 4. Simulate the eep1 sheet of lab3i running `int-test1.ram` in the waveform simulator. View IRQ1, IRQ1reset, NZCVout, and PCX in addition to PC, INS, R0, and R2. Trace execution from the moment IRQ1 is asserted through the interrupt jump, ISR execution, and the RETINT return to the base program.





When interrupt happen, PC goes to 0xf00 and starts. Since our first interrupt is at about 20th cycle, so initially, PC runs in main program and R0 increases by 1 each cycle.



After the 20th cycle, IRQ1 is high so interrupt happens.

1. R0, R1, and flags are stored into RAM.
2. Run the interrupt program, which increments R2 by 1 after 10 cycles.
3. Finish interrupt and load back R0, R1, flags values.
4. Continue to run main program, which is ADD R0, #1.

Using the code and waveforms, explain why the base program PC, flags, and R0 remain unaffected by ISR execution.

Because they are stored into RAM before interrupts, and loaded back after that.

4.3 Multiple interrupt sources

The program int-test2.txt demonstrates handling two interrupt sources simultaneously. Both sources are ORed onto the single interrupt request input (IRQ) of the eep1 sheet. When an interrupt occurs, the ISR software determines which source is active, clears it, performs the appropriate action, and returns.

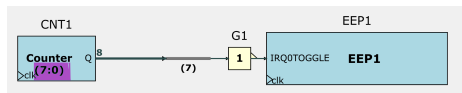
```
int-test2.txt
// test interrupt 0 & 1
// interrupts are not nested,
// irq has conditional jump to determine which interrupt
// is taken.
// Base program
SETI // set FFIDN.Q (enable interrupts)
LOOP INC LOOP
// code from here goes into address 0xf00
STR R0, {BufF00} // save base program R0
MOV R0, #0 // save base program flags
CLR R0, {BufF01}
CLR R0, {BufF02}
AND R0, #2 // is IRQ1 active?
// if so jump to ISR1
// otherwise, ISR0 must be active.
ISR0 MOV R0, #1
STR R0, {BufF01} // Interrupt 0 reset high
MOV R0, #0
STR R0, {BufF02} // Interrupt 0 reset low
// Payload for Interrupt 0
// increment R0 (BufF02) by 1
ADD R0, #1
// next loop to add some time to INT0
MOV R0, #1
CLR R0, #1 // Give some delay
JNE --1
// end of payload
INTEN0
CLR R0, {BufF01}
MOV R0, #0 // restore base program flags
CLR R0, {BufF02} // restore R0
RETINT
ISR1 MOV R0, #2
STR R0, {BufF01} // Interrupt 1 reset high
MOV R0, #0
STR R0, {BufF02} // Interrupt 1 reset low
// Payload for Interrupt 1
ADD R0, #1
// end of payload
INTEN1
CLR R0, {BufF01}
MOV R0, #0 // restore base program flags
CLR R0, {BufF02} // restore base program R0
RETINT // return to base program and set FFIDN
```

This test runs the same main program.

When there is an interrupt, PC will run an AND R0, #2 to check whether it is ISR0 or ISR1.

We can distinguish them by finding whether it is R3 or R4 increments by 1. After that, it loads back parameters in main program and continues.

In this test, IRQ0TOGGLE is driven by a counter in the top-level eep1x sheet to generate an INTO interrupt approximately every 128 cycles, starting at cycle 0.



This design sends 7th bit of counter result to IRQ0TOGGLE, and it toggles every 128 cycles.

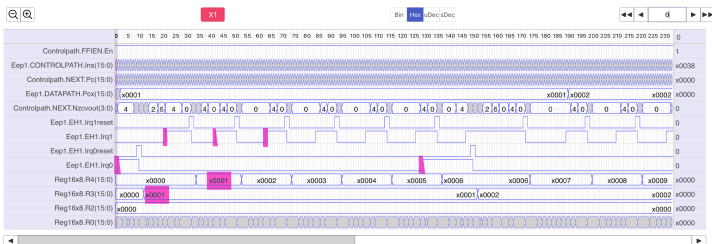
Task 5. Simulate the eep1x (not eep1) sheet of lab3i running int-test2.ram in the waveform simulator. Refer to the assembler listing in int-test2.txt. View FFIDN, IRQ0, IRQ0reset, IRQ1, IRQ1reset, NZC.Vout, and PCX in addition to PC, INS, R0, R2, and R3.

So now we know that IRQ0TOGGLE is in charge of ISR0, which happens every 128 cycles.

While ISR1 is set by the hardware to happen every 20 cycles.

Note that this program cannot handle with nested interrupts. So for example, when we are doing ISR0, and ISR1's signal is toggled, it won't be seen.

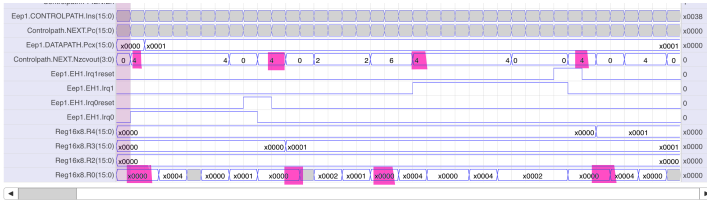
From the first 50 clock cycles determine which cycles execute the base program, which execute INTO servicing, and which execute INT1 servicing. Confirm that after each interrupt return, R0 and the N.Z.C.V flags remain unchanged.



So in cycle 1, 128: ISR0 executes and IRQ0 = 1; R3 increments by 1.

In cycle 20, 40: ISR1 executes and IRQ1 = 1; R4 increments by 1.





We can see that after each interrupt, the values of R0 (0x0000) and the flags (4) are written back, and these values are the same as they were immediately before the interrupt occurred.

Task 6. Simulate the eep1x sheet of lab3i running int-test3.ram in the waveform simulator, referring to the assembler in int-test3.txt.

```
// int-test3.txt
// test interrupt 0 & 1 - code for nested interrupts
// Interrupt 1 has higher priority than interrupt 0
// The ISR starts with a conditional jump as
// int-test2
//
// Base program
SETI // set FFIDN,0 (enable interrupts)
ADD R0, #1 // increment R0
JNE LOOP
Main
ORG 0x0000 // code from here goes into address 0x0000
// this code assumes that base program does not use R0, R1
// therefore, unlike int-test2, these registers can be used to hold
// temporary values, this change is made to make the demo shorter.
// otherwise the extra code here would take longer to trace through.
// otherwise memory would need to be used for temp variables
// this is possible, but would require more instructions
// save base program flags temporarily
MOV R6, R0 // save R0 temporarily
MOV R5, R1 // save R1 temporarily
// ISR condition check
JNE ISR1 // if so, jump to ISR1
// otherwise, ISR0 must be active.
ISR0
MOV R0, #0 // Interrupt 0 reset high
MOV R0, #0 // Interrupt 0 reset low
STR R5, [R0FFFF] // save base program R5 somewhere else
STR R6, [R0FFFF] // save base program R6 somewhere else
MOV R0, PCX
STR R0, [R0FFFF]
// Payload for interrupt 0
// finally it is possible to allow a nested interrupt
// add some time to INT0
ADD R0, #1
JNE -1
// end of payload
INT0EN
LDR R0, [R0FFFF] // restore base program R0
MOV R0, R5 // restore base program R5
CLR R0 // can't allow INT1 after this since it changes PCX
LDR R0, [R0FFFF]
MOV PCX, R0 // restore base program PCX
LDR R0, [R0FFFF] // restore base program R0
RETI
ISR1
// nothing can interrupt INT1
MOV R0, #0
STR R0, [R0FFFF] // Interrupt 1 reset high
MOV R0, #0 // Interrupt 1 reset low
STR R0, [R0FFFF] // Interrupt 1 reset low
// Payload for interrupt 1
ADD R0, #1
// end of payload
INT1EN
MOV R0, R6 // restore base program R6
MOV R0, R5 // restore base program R5
RETI // return to base program and set FFIDN
LOOP
```

This is wrong because MOV instruction will change flags state, so flags will be overwritten and lost.
Instead, we should save flags first.
Like:
MOV R6, Flags
MOV R5, R0

Now we can implement nested interrupts, as every time when ISR0 is running, it will write PCX into RAM. PCX is the current PC in the main program. So location in main program will not be lost.
One thing need to be noticed is that we can only implement ISR1 nested to ISR0, not the reverse, as ISR1 doesn't have function of writing PCX.

Note the placement of SETI and CLRI inside the INT0 ISR. What do these instructions control? Explain why correct nesting of INT1 inside INT0 requires both instructions, and why SETI cannot be placed earlier.

```
// int-test3.txt
// test interrupt 0 & 1 - code for nested interrupts
// Interrupt 1 has higher priority than interrupt 0
// The ISR starts with a conditional jump as
// int-test2
//
// Base program
SETI // set FFIDN,0 (enable interrupts)
ADD R0, #1 // increment R0
JNE LOOP
Main
ORG 0x0000 // code from here goes into address 0x0000
// this code assumes that base program does not use R0, R1
// therefore, unlike int-test2, these registers can be used to hold
// temporary values, this change is made to make the demo shorter.
// otherwise the extra code here would take longer to trace through.
// otherwise memory would need to be used for temp variables
// this is possible, but would require more instructions
// save base program flags temporarily
MOV R6, R0 // save R0 temporarily
MOV R5, R1 // save R1 temporarily
// ISR condition check
JNE ISR1 // if so, jump to ISR1
// otherwise, ISR0 must be active.
ISR0
MOV R0, #0 // Interrupt 0 reset high
MOV R0, #0 // Interrupt 0 reset low
STR R5, [R0FFFF] // save base program R5 somewhere else
STR R6, [R0FFFF] // save base program R6 somewhere else
MOV R0, PCX
STR R0, [R0FFFF]
// Payload for interrupt 0
// finally it is possible to allow a nested interrupt
// add some time to INT0
ADD R0, #1
JNE -1
// end of payload
INT0EN
LDR R0, [R0FFFF] // restore base program R0
MOV R0, R5 // restore base program R5
CLR R0 // can't allow INT1 after this since it changes PCX
LDR R0, [R0FFFF]
MOV PCX, R0 // restore base program PCX
LDR R0, [R0FFFF] // restore base program R0
RETI
ISR1
// nothing can interrupt INT1
MOV R0, #0
STR R0, [R0FFFF] // Interrupt 1 reset high
MOV R0, #0 // Interrupt 1 reset low
STR R0, [R0FFFF] // Interrupt 1 reset low
// Payload for interrupt 1
ADD R0, #1
// end of payload
INT1EN
MOV R0, R6 // restore base program R6
MOV R0, R5 // restore base program R5
RETI // return to base program and set FFIDN
LOOP
```

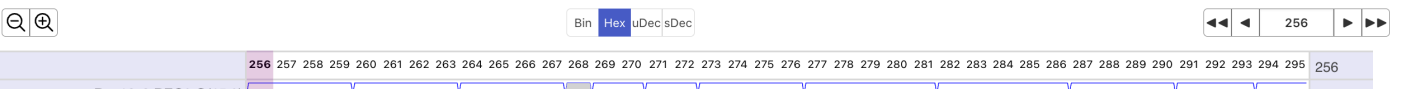
EEP1 "Extra MOV" Coding	INS (8)	C (4:2)	A (11:9)	B (7:5)	INS Alternative Mnemonic	Operation	Notes
MOV R6, R0, R0	0	6	0	0	0x0018 RETINT	PC ← PCX; Flags := 1	Return from interrupt, enable interrupts
MOV R6, R0, R1	0	6	0	1	0x0038 SETI	Flags := 1	Allow (enable) interrupts
MOV R6, R0, R2	0	6	0	2	0x0058 CLRI	Flags := 0	Disable interrupts
MOV R7, Ra, R0	0	7	a	0	MOV Flags, Ra	NZCV := Ra[3:0]	Restore flags from register (no write to Ra)
MOV R7, Ra, R1	0	7	a	1	MOV PCX, Ra	PCX := Ra	Restore PCX from register (no write to Ra)
MOV R7, Ra, R2	0	7	a	2	MOV Ra, Flags	Ra[3:0] := NZCV	Save flags to register
MOV R7, Ra, R3	0	7	a	3	MOV Ra, PCX	Ra := PCX	Save PCX to register

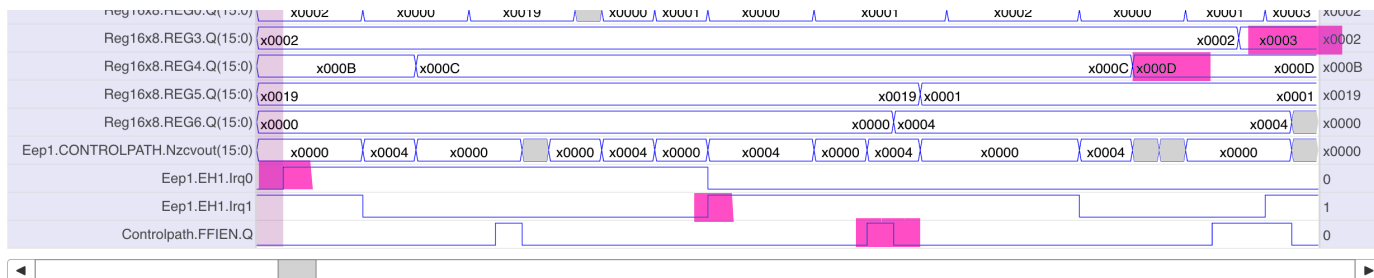
Figure 7: EEP1I Instructions to support interrupts

SETI and CLRI are enables. In ISR0, SETI is placed only after PCX, the flags, and the required register values have been saved, so that a higher-priority IRQ1 can safely nest inside IRQ0.
CLRI is placed before restoring the saved context, especially before restoring PCX, to prevent another interrupt from corrupting the return address.
If SETI were moved earlier, nested interrupts could occur before the original context was saved, so information from the interrupted code could be lost and the program would not return correctly.

Explain why PCX must be saved and later restored in the INT0 ISR here, whereas this was not necessary in int-test2.txt.

Because now it is nested interrupt, if ISR1 is added on ISR0, PC in main program will be lost. So we need additional steps to store and load PCX.
Note that even if we don't nest ISR1, the program still stores PCX.





So within this simulation, it is obvious that:

1. 256th cycle, IRQ0 = 1. ISR0 starts.
2. 272th cycle, IRQ1 = 1. ISR1 interrupt signal is raised.
3. 278th cycle, FFIEN = 1 and IRQ1 is written, which means nested interrupt of ISR1 starts.
4. 288th cycle, R4 increments by 1, which means ISR1 is running.
5. 292th cycle, R3 increments by 1, which means ISR1 ended, and ISR0 continues to run.

This shows a standard nested interrupts correctly.